PROJECT RESILIENCE-ALPHA

Xijin Pink

Une plateforme de dons fictive face à un incident de sécurité

Dossier d'ingénierie et de gouvernance

Groupe G2

Julien Demogue	Services et API
Louis Marec	IA et données
Eric Girard	Infrastructure
Noé Filippi	Frontend
Sterenn	Frontend

Octobre 2026. Toutes les données utilisées sont fictives. Les essais offensifs ont été menés uniquement sur nos propres machines, à travers le hotspot RESILIENCE-G2.

Synthèse

Xijin Pink est un prestataire de services de paiement fictif qui collecte des dons pour Octobre Rose. Nous l'avons construit pendant la semaine du workshop pour répondre à trois questions : quand une attaque frappe un service de paiement, est-ce qu'on la voit, est-ce qu'on comprend ce qui se passe, et est-ce que le service redevient utilisable ?

La plateforme tourne entièrement sur le PC Hôte avec Docker Compose. Elle comprend un site de dons en React, trois services Node.js (authentification, paiement, prestataire de paiement simulé), une base MariaDB, un WAF Nginx qui sert de seule porte d'entrée, une chaîne d'observabilité Prometheus, Loki et Grafana, un détecteur d'anomalies par apprentissage automatique et un assistant RAG local. Pour la consigne additionnelle, nous avons ajouté une authentification par reconnaissance faciale reliée à un voyant ESP8266. L'ensemble démarre avec une seule commande.

Sur une campagne d'attaques réelle (déni de service, bourrage d'identifiants, aspiration, injection, exfiltration), le détecteur a repéré les cinq catégories en 10 à 15 secondes, avec une AUROC proche de 0,95. Le WAF a renvoyé 4 888 réponses 429 pendant un flood, et les 42 tests unitaires du dépôt passent. Les sections 9 à 11 relient ces choix techniques à l'analyse EBIOS RM, au PCA/PRA et aux exigences NIS2 et DORA.

Sommaire

1. Contexte et objectifs 3
2. Architecture et flux 3
3. Décisions structurantes 4
4. Nature de l'incident 5
5. Détection 6
6. Reprise du service 7
7. Résultats des essais 8
8. Sécurité 8
9. Analyse de risques EBIOS RM 9
10. PCA / PRA 11
11. NIS2 et DORA 12
12. Contributions de l'équipe 13
13. Limites et suites 13
Annexe A. Renvois vers le dépôt 14
Annexe B. Démarrer et vérifier la plateforme 14

1. Contexte et objectifs

Le sujet nous place dans la peau de l'équipe technique d'un prestataire de paiement. Le système doit subir un incident de sécurité contrôlé, le détecter et revenir à un état utilisable sans réparation à la main. Le jury doit pouvoir suivre l'état du service avant, pendant et après l'incident.

Dès le premier jour, nous avons fixé une définition de la reprise qui a guidé le reste du projet. Un conteneur qui redémarre ne prouve rien : l'API peut rester muette, un paiement peut échouer, une transaction peut rester bloquée. Nous considérons le service comme rétabli quand l'API répond, qu'un nouveau don fictif atteint le statut `paid` et que les données restent cohérentes.

Périmètre

Le laboratoire compte trois postes. Le PC Hôte fait tourner toute la plateforme, le PC Attaquant lance les scénarios d'attaque, et un PC IA facultatif peut héberger Ollama. Tout passe par le point d'accès Windows `RESILIENCE-G2` (réseau 192.168.137.0/24, hôte en 192.168.137.1). Il n'y a ni environnement public ni préproduction. Les réseaux du campus, les autres équipes et les services Internet sont hors périmètre. Les données sont fictives : adresses de test, montants inventés, carte de test 4242 4242 4242 4242.

Ce dossier présente les décisions et les résultats. Le code, les contrats d'interface et les procédures détaillées restent dans le dépôt, livré dans l'archive `Workshop2026-M2-G2-Code.zip`. Chaque section indique les chemins à consulter et l'annexe A en donne la liste complète.

2. Architecture et flux

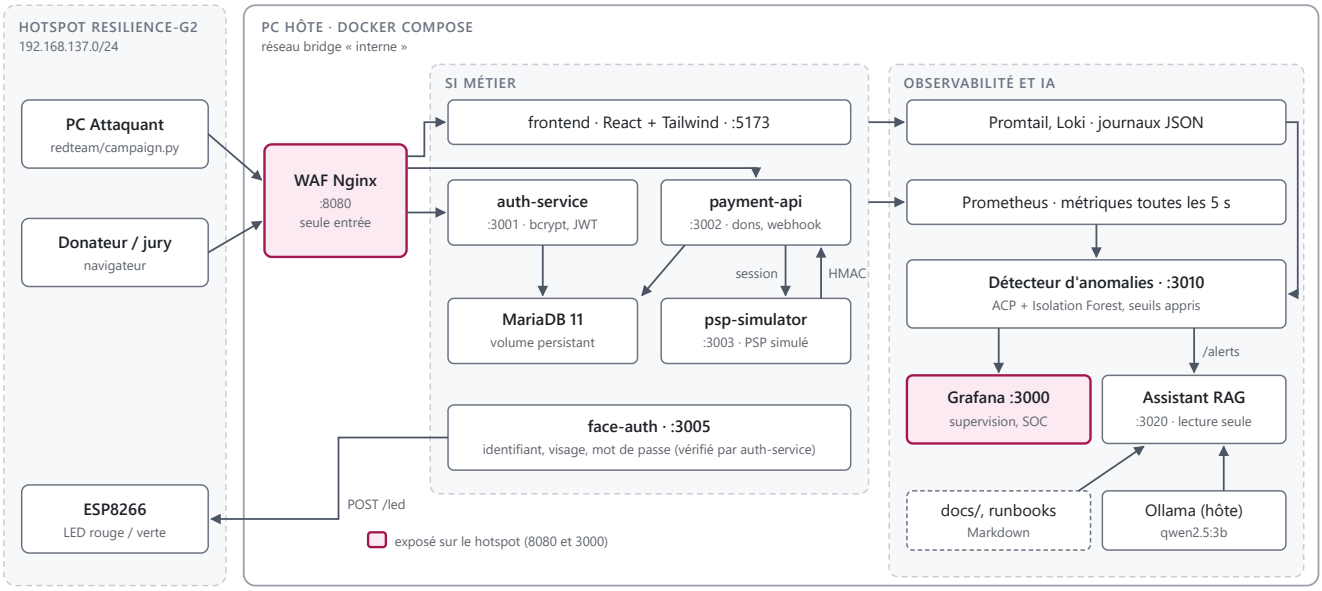


Figure 1. Architecture déployée sur le PC Hôte. Grafana lit Prometheus et Loki ; le détecteur lit les journaux et les métriques puis publie ses alertes.

2.1 Rôle des composants

Composant	Rôle	Technologie
`waf`	Seule entrée HTTP : routage vers le site et les API, filtrage, limitation de débit, journal d'accès JSON.	Nginx 1.27
`frontend`	Site de dons : accueil, don, retour de paiement, classement, historique, profil, connexion et inscription.	React, Vite, Tailwind
`auth-service`	Comptes, connexion, profil, émission des jetons JWT.	Node.js, Express
`payment-api`	Dons et transactions, historique, classement, titres de donateur, réception du webhook du PSP.	Node.js, Express
`psp-simulator`	Prestataire de paiement simulé : session de paiement, page de confirmation, webhook signé.	Node.js, Express

Composant	Rôle	Technologie
`mariadb`	Tables `users`, `transactions` et `titles`.	MariaDB 11
`promtail`, `loki`, `prometheus`	Collecte et stockage centralisé des journaux de tous les conteneurs ; métriques des services toutes les 5 secondes, gardées 7 jours.	Loki 3.1, Prometheus 2.54
`grafana`	Tableaux de supervision et de SOC, chargés depuis le dépôt au démarrage.	Grafana 11.2
`anomaly-detector`	Détection d'anomalies par apprentissage, alertes, métriques de score.	Python, scikit-learn, FastAPI
`rag-assistant`	Assistant opérateur sur la documentation et les runbooks.	Python, Ollama
`face-auth`	Authentification faciale (enrôlement, identification, contrôle du mot de passe) et back-office.	Python, OpenCV
`esp8266-led`	Voyant d'état de l'authentification, sur le hotspot.	ESP8266, PlatformIO

Pour le développement, le fichier compose publie aussi les ports des services (3001 à 3003), de phpMyAdmin (8081), de Prometheus (9090) et de Loki (3100). Pendant la démonstration, seuls 8080 et 3000 doivent rester ouverts sur le hotspot ; les autres se ferment dans le pare-feu de l'hôte.

2.2 Parcours d'un don

1. Le donateur choisit un montant, puis saisit la carte de test. Le formulaire contrôle la carte dans le navigateur et aucune donnée de carte n'est envoyée au serveur.
2. Le site appelle `POST /api/donations/checkout` à travers le WAF. `payment-api` valide le montant (en centimes), la devise et l'adresse e-mail si le don est anonyme, enregistre une transaction `pending` et ouvre une session chez le PSP simulé.
3. Le navigateur est redirigé vers la page de paiement du PSP, où le donateur confirme (ou simule un refus).
4. Le PSP appelle `POST /api/donations/webhook` avec un corps signé en HMAC-SHA256 dans l'en-tête `x-signature-psp`.
5. `payment-api` recalcule la signature, la compare en temps constant et passe la transaction à `paid` ou `failed`. Une signature invalide est rejetée (401) et journalisée comme alerte.
6. Le donateur revient sur le site. Ses dons payés alimentent son historique, le classement et son titre de donateur.

La base ne garde que le nécessaire : e-mail, montant, devise, statut, références et horodatages. Un donateur connecté est rattaché à son compte, sinon l'e-mail suffit. Le détail est dans `docs/payment-flow.md` et chaque service publie sa spécification OpenAPI sur `/api-docs`.

2.3 Flux de supervision

Les trois services Node.js partagent le même middleware de télémétrie (`src/middleware/telemetry.js`). Il attribue un `request_id` à chaque requête, écrit une ligne JSON `http_access` (méthode, modèle de route, statut, durée, octets, IP cliente) et expose des métriques `prom-client` sur `/metrics`. Les événements métier et de sécurité (`checkout_created`, `webhook_received`, `webhook_invalid_signature`, `payment_confirmed`, `login_failed`, `validation_rejected`) portent le même identifiant, ce qui permet de rattacher un effet à la requête qui l'a produit. Le WAF écrit son propre journal JSON et transmet la vraie IP du client dans `X-Forwarded-For`.

Promtail lit les journaux des conteneurs et les pousse vers Loki. Prometheus interroge les `/metrics` toutes les 5 secondes. Aucun journal ne contient de corps de requête, de jeton, de mot de passe ni le secret du webhook. Le contrat complet est décrit dans `docs/api/telemetry-contract.md`.

3. Décisions structurantes

Décision	Pourquoi	Ce que nous acceptons
Simuler le PSP dans la plateforme plutôt qu'utiliser Stripe en mode test	Pas de dépendance à Internet pendant la démo et le clip en une prise, aucun tiers dans le périmètre d'attaque. La forme réelle est conservée : session de paiement puis webhook signé.	Ni relance du webhook ni 3-D Secure ; sessions gardées en mémoire.
Une seule base MariaDB pour auth-service et payment-api	Simple à exploiter en quatre jours, une seule base à protéger et à sauvegarder.	Couplage entre les deux services. Une base par service serait plus propre.

Décision	Pourquoi	Ce que nous acceptons
WAF Nginx écrit à la main plutôt que ModSecurity avec l'OWASP CRS	Règles lisibles, image légère, journal JSON adapté au SOC.	Couverture plus étroite que le CRS. La bascule est décrite dans `services/waf/README.md`.
Détecter d'abord à partir du journal du WAF	Un service qui tombe cesse d'écrire. Le WAF, lui, continue de voir les 502 et les 504.	Si le WAF tombe, la détection perd sa source principale.
Apprendre le comportement normal et en déduire le seuil	Exigence du sujet, et c'est ce qui permet de repérer une attaque jamais vue.	Il faut un trafic normal réaliste, erreurs ordinaires comprises.
Prometheus, Loki et Grafana plutôt qu'Elastic	Le PC Hôte a peu de mémoire et le sujet écarte Elastic.	Recherche plein texte plus limitée.
Supervision dans Grafana, tableaux versionnés dans le dépôt	Chargés automatiquement au démarrage, ils lisent directement Prometheus et Loki, sans code à maintenir.	Mise en page moins libre qu'une application dédiée.
Assistant RAG local sur Ollama avec `qwen2.5:3b`	Tient dans 8 Go, rien ne sort du labo, bon niveau en français.	Réponses parfois approximatives, d'où les citations de sources.
Secrets générés au premier démarrage dans un `.env` ignoré par Git	Aucun secret dans le dépôt et un démarrage en une commande.	Le fichier reste en clair sur l'hôte.
Démarrage en une commande (`scripts/start.sh` ou `start.ps1`)	L'équipe et le jury relancent la même plateforme de la même façon.	Premier lancement long (construction des images, modèles).

4. Nature de l'incident

4.1 Scénarios d'attaque

Les attaques sont des scripts Python reproductibles (`redteam/attacks.py`) lancés depuis le PC Attaquant contre le WAF. Leur trafic porte trois adresses sources (192.168.137.201 à 203, via `X-Forwarded-For`), distinctes de celles des utilisateurs normaux. L'intensité est volontairement bornée pour tourner sur un ordinateur portable.

Catégorie	Cible	Technique	Effet observable
`availability`	`POST /api/donations/checkout`	Flood de 40 clients concurrents pendant 60 s, avec montée progressive.	Débit et latence qui s'envolent, erreurs 5xx, réponses 429 du WAF.
`auth-abuse`	`POST /api/auth/login`	Bourrage d'identifiants depuis quelques adresses.	Rafale de 401 et d'événements `login_failed`.
`scraping`	`GET /api/donations/leaderboard`	Énumération à haute fréquence par un robot.	Débit concentré sur une route et une adresse.
`injection`	Entrées de connexion et de paiement	Vraies charges SQL, XSS et traversée de répertoire.	Rafale de 400 (validation) et de 403 (WAF).
`exfiltration`	Historique authentifié et classement	Lecture massive depuis une adresse.	Volume de données sortantes anormal.

4.2 L'incident mis en avant

Pour la démonstration, nous mettons en avant le déni de service sur le paiement. C'est l'incident dont l'effet métier se voit le mieux : la latence grimpe, des dons n'aboutissent plus, puis le service revient. Son critère de reprise est aussi le plus net, puisqu'il suffit de refaire un don. Les quatre autres scénarios servent à entraîner et à évaluer le détecteur, et montrent que la chaîne ne se limite pas à un seul type d'attaque.

Les sondes d'injection ne peuvent pas aboutir, parce que les requêtes SQL sont paramétrées et les entrées validées. Leur intérêt est ailleurs : elles laissent une empreinte (rejets en série, routes variées, outil reconnaissable) que le détecteur apprend à repérer et que le WAF bloque en partie avant qu'elle n'atteigne les services.

4.3 Campagne et étiquetage

`redteam/campaign.py` commence par du trafic normal seul, qui sert de référence « avant ». Chaque attaque est ensuite injectée pendant que le trafic normal continue, avec des pauses entre les épisodes, puis vient une phase de retour au calme. Le script écrit un manifeste avec l'heure de début et de fin de chaque épisode. Le trafic normal vient de

`tools/traffic/normal_traffic.py`, qui joue de vrais parcours (inscription, connexion, don complet, consultation) et commet les erreurs ordinaires d'un utilisateur : mauvais mot de passe, montant invalide, paiement abandonné. Sans elles, le modèle apprendrait que tout 401 est une attaque, ce qui reviendrait à un seuil codé en dur. Les journaux sont ensuite exportés de Loki (`tools/dataset/export_loki.py`) et chaque fenêtre est étiquetée d'après le manifeste.

5. Détection

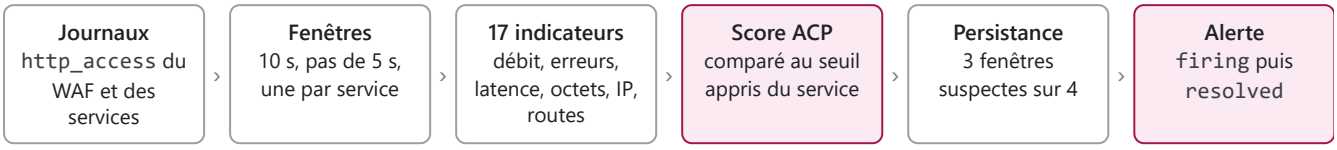


Figure 2. Chaîne de détection, de la ligne de journal à l'alerte.

5.1 Des journaux aux observations

Le détecteur ne juge pas les requêtes une par une. Une attaque est un comportement qui dure : un flood, une série d'échecs de connexion, une aspiration méthodique. L'unité d'analyse est donc un service (`waf`, `payment-api`, `auth-service`, `psp-simulator`) sur une fenêtre de 10 secondes, recalculée toutes les 5 secondes. Pour chaque fenêtre, nous calculons 17 indicateurs (FeatureSpec v2) : nombre et débit de requêtes, taux d'erreurs 4xx et 5xx, latences médiane et p95, octets sortants (p95, somme, moyenne par requête), nombre d'IP distinctes et part de la première, nombre de routes distinctes et part de la première, taux d'échecs d'authentification, signatures de webhook invalides, et enfin CPU et mémoire du service lus dans Prometheus. Les indicateurs à forte dispersion (volumes, latences, octets) passent par un log1p avant la standardisation, pour qu'un pic ordinaire ne ressemble pas à une attaque. Les appels techniques sur `/metrics` et `/health` sont exclus.

Nous ne calculons pas d'observation par adresse IP. Dans nos essais, tout le trafic normal vient d'une même machine et « normal pour une IP » n'aurait pas de sens. Les adresses dominantes apparaissent en revanche dans les éléments de preuve de l'alerte, pour qu'on puisse les bloquer au WAF.

5.2 Modèle à deux étages

Le premier étage est non supervisé : il n'a vu que du trafic normal. Les indicateurs sont standardisés, puis une analyse en composantes principales (ACP) conserve 95 % de la variance. Le score d'anomalie est l'erreur de reconstruction : plus une fenêtre s'éloigne de ce que le modèle sait reconstruire, plus il monte. L'erreur indicateur par indicateur fournit directement l'explication de l'alerte (valeur observée face à la valeur attendue). Un Isolation Forest de 200 arbres, entraîné sur les mêmes données, sert de point de comparaison.

Le seuil n'est jamais choisi à la main. Il correspond au 95e percentile des scores obtenus sur un lot de trafic normal mis de côté, et il est calculé service par service, avec un seuil global en secours. Un service calme comme auth-service garde ainsi un seuil bas, et une rafale de 401 n'y est pas noyée dans la variance d'un service plus bruyant.

Le second étage est un classifieur supervisé (Gradient Boosting) entraîné sur les fenêtres étiquetées. Il propose la catégorie probable parmi les cinq de la campagne. En dessous de 0,5 de confiance, il s'abstient et répond `unknown`, ce qui oriente vers la procédure générique. Apprentissage et test sont séparés par campagne et non au hasard, car deux fenêtres voisines se ressemblent trop pour qu'un tirage aléatoire donne une mesure honnête.

5.3 Cycle de vie d'une anomalie

Une fenêtre suspecte isolée ne déclenche rien. Le service passe en attente (PENDING) dès la première, puis en alerte (FIRING) quand 3 des 4 dernières fenêtres sont suspectes. Il revient à la normale (RESOLVED) après 3 fenêtres normales consécutives. Avec un pas de 5 secondes, une attaque franche déclenche l'alerte en 10 à 15 secondes, ce qui laisse le temps de montrer toute la chaîne dans un clip de 60 secondes. Un service sans trafic est considéré comme normal : l'absence de requêtes n'est pas une anomalie en soi, et la panne d'un service se lit de toute façon dans les 502 et 504 du WAF.

Le retour à RESOLVED compte autant que le déclenchement. C'est un composant indépendant qui confirme que le comportement est redevenu normal, ce qui complète le test métier (un nouveau don payé) et va plus loin que « le conteneur tourne ».

5.4 Contenu et diffusion d'une alerte

À chaque changement d'état, `ai/anomaly-detection/live_scorer.py` produit une alerte JSON conforme à `docs/api/alert-contract.md` : identifiant stable, statut `firing` ou `resolved`, service concerné, score et seuil, sévérité (haute au-delà de deux fois le seuil), catégorie et confiance, indicateurs les plus déviants, version du modèle. Elle est écrite dans le journal, donc visible dans Loki et dans le SOC, conservée sur `/alerts` pour l'assistant RAG, et envoyée en POST à l'adresse `REMEDIATION_URL` quand celle-ci est configurée. Le détecteur publie aussi `anomaly_score`, `anomaly_threshold`, `anomaly_state` et `anomaly_alerts_total` pour Grafana.

5.5 Rendre visibles l'avant, le pendant et l'après

Deux tableaux Grafana sont provisionnés au démarrage. « Supervision & Résilience » affiche le débit, le taux de 5xx, la latence p95 et la mémoire de chaque service, puis le score d'anomalie face au seuil et la marge entre les deux. Tant que la marge reste sous zéro, le service est normal ; pendant l'incident, on la voit passer au-dessus puis redescendre. « SOC / SIEM » regroupe les blocages (403, 405) et limitations (429) du WAF, le nombre de services en alerte, les échecs de connexion et les signatures de webhook invalides par minute, et un journal de sécurité qui croise WAF, authentification et alertes du détecteur.

5.6 Assistant opérateur (RAG)

L'assistant (`ai/rag-assistant/`) répond aux questions d'un opérateur à partir de la documentation du dépôt et des runbooks, c'est-à-dire les fichiers Markdown de `docs/`. Tout tourne en local sur Ollama : `nomic-embed-text` pour les vecteurs, `qwen2.5:3b` pour la génération, bien en dessous de la limite de 8 milliards de paramètres. La route `/explain` récupère la dernière alerte du détecteur, oriente la recherche vers le runbook de sa catégorie et répond en citant ses sources. L'alerte est injectée comme donnée non fiable, car ses champs peuvent contenir des chaînes venues d'un attaquant, et le prompt système interdit de les traiter comme des instructions. L'assistant ne fait que lire : il n'a ni outil, ni accès au système, ni secret, et ce n'est pas lui qui pilote la remédiation.

6. Reprise du service

6.1 Mécanismes en place

La première ligne est l'absorption à l'entrée. Le WAF limite chaque adresse à 20 requêtes par seconde avec une marge de 40, et répond 429 au-delà. Un flood venu de quelques adresses est donc freiné avant d'atteindre `payment-api`, et les autres donateurs continuent d'être servis.

Viennent ensuite les mécanismes de Docker. Tous les conteneurs sont déclarés en `restart: unless-stopped` : un service qui plante redémarre seul, y compris après un redémarrage de l'hôte. MariaDB a un contrôle de santé (toutes les 10 secondes, 10 essais) et les services qui en dépendent attendent qu'elle soit prête avant de démarrer.

La plateforme sait aussi repartir d'un clone neuf sans intervention. Si aucun modèle n'existe au premier démarrage, le détecteur en entraîne un sur des données synthétiques. L'assistant RAG démarre même quand Ollama n'est pas prêt et construit son index à la première question. `scripts/verify.sh` contrôle ensuite les onze points de santé de la plateforme.

Côté données, les transactions sont écrites dans InnoDB sur un volume persistant, si bien qu'un redémarrage de conteneur ne perd rien de ce qui a été validé. La mise à jour par webhook est idempotente : rejouer la même confirmation laisse la transaction dans le même état.

6.2 Réponse attendue selon le type d'incident

Rétablir la disponibilité et corriger une compromission sont deux problèmes différents. Après un déni de service, il suffit que le service réponde de nouveau et qu'un don passe. Après une injection ou une exfiltration, il faut contenir, vérifier l'intégrité des données et révoquer ce qui doit l'être ; un service qui répond ne dit rien de l'état des données. Le contrat d'alerte associe donc à chaque catégorie une réponse et une preuve de rétablissement propres, détaillées dans les runbooks de `docs/runbooks/`.

Catégorie	Réponse attendue	Preuve de rétablissement
`availability`	Resserrer la limite du WAF pour les adresses en cause, redémarrer ou dupliquer `payment-api`.	Un nouveau don atteint `paid`.
`auth-abuse`	Bloquer les adresses sources au WAF, limiter `/login`.	Un compte légitime se connecte.

Catégorie	Réponse attendue	Preuve de rétablissement
`injection`	Durcir la règle WAF, bloquer l'adresse, contrôler l'intégrité, restaurer en cas de doute.	Contrôle d'intégrité réussi et don payé.
`exfiltration`	Contenir en bloquant l'adresse, révoquer les jetons concernés.	Plus de volume sortant anormal.
`unknown`	Procédure générique : redémarrer la cible, resserrer le WAF.	Don payé et score stable sous le seuil.

Chaque runbook prévoit un cas d'escalade. Si le rétablissement échoue deux fois de suite, ou si une trace montre qu'une charge a réellement abouti (erreur SQL dans les journaux par exemple), l'opérateur arrête et prévient un coach.

7. Résultats des essais

Essai	Méthode	Résultat
Tests unitaires des services Node.js	`node --test`, base et PSP remplacés par des doublures	auth-service 10/10, payment-api 12/12, psp-simulator 7/7
Tests unitaires du détecteur	`unittest` sur données synthétiques	8/8
Tests de l'authentification faciale	`unittest` : réessai, blocage progressif, calcul de similarité	5/5
Détection sur données réelles	Campagne complète sur la plateforme, apprentissage et test sur des campagnes distinctes	5 catégories sur 5 détectées, alerte 10 à 15 s après le début de l'attaque, AUROC ≈ 0,95, exactitude du classifieur de catégorie ≈ 0,93
WAF	Routage, requêtes malveillantes, flood	Signatures bloquées en 403, 4 888 réponses 429 pendant le flood
Démarrage en une commande	`start.sh` puis `verify.sh`, y compris depuis un clone neuf	11 points de santé sur 11
Reconnaissance faciale	Photos réelles, même personne et personnes différentes	Similarité ≈ 0,74 pour une même personne, ≈ 0,10 entre personnes différentes (seuil 0,363)

Les tests unitaires ont été relancés le 22 octobre 2026 sur la branche principale. Les autres résultats sont consignés dans l'historique Git (commits aa6ef0b, 5a846dc et a1c582d) et dans `services/face-auth/README.md`.

Le délai de détection correspond à ce que la conception prévoit : trois fenêtres suspectes avec un pas de 5 secondes. Une AUROC de 0,95 indique que le score sépare bien le trafic normal du trafic d'attaque. Ces chiffres restent ceux d'une campagne jouée sur un seul ordinateur portable, et nous ne les présentons pas comme une performance générale. Une mesure manque encore : le nombre de fausses alertes par heure sur du trafic purement normal. C'est elle qui dira si le détecteur est supportable en exploitation. Côté reconnaissance faciale, l'écart entre 0,74 et 0,10 laisse une marge confortable autour du seuil de 0,363, mais il n'a été mesuré que sur quelques visages.

8. Sécurité

8.1 Défense en profondeur

Couche	Mesures	Où le vérifier
Réseau	Un réseau Docker interne unique. La base, le PSP, Loki, Prometheus et le moteur Docker ne sont pas destinés au hotspot. Promtail accède au socket Docker en lecture seule.	`infra/docker/docker-compose.yml`
Entrée	WAF comme seule entrée : limitation de débit par adresse, blocage des motifs de traversée, SQLi, XSS et Log4Shell dans le chemin et les paramètres (403), méthodes non standard refusées (405), corps limité à 1 Mo, délai de 10 s, version du serveur masquée, en-têtes `X-Frame-Options`, `nosniff` et `Referrer-Policy`.	`services/waf/nginx.conf`
Application	Validation de toutes les entrées (e-mail, montant, devise, longueurs), requêtes SQL paramétrées, mots de passe hachés avec bcrypt (10 tours, 8 caractères minimum), jetons JWT signés, même message d'erreur que le compte existe ou non.	`services/*/src/`

Couche	Mesures	Où le vérifier
Paiement	Webhook signé en HMAC-SHA256 et comparé en temps constant, signatures invalides rejetées et remontées au SOC. Aucune donnée de carte ne quitte le navigateur.	`services/payment-api/src/signature.js`
Journaux	JSON structuré, corrélé par `request_id`, sans corps, jeton, mot de passe ni secret. Événements d'audit à la création du paiement et à sa confirmation.	`src/middleware/telemetry.js`
Secrets	Générés au premier démarrage (24 octets aléatoires par `openssl rand`) dans un `.env` ignoré par Git. Aucun secret dans le dépôt.	`scripts/setup.sh`
IA	Assistant en lecture seule, sans outil, alerte traitée comme donnée non fiable.	`ai/rag-assistant/`

8.2 Fonction SOC / SIEM

Le SOC s'appuie sur Loki et sur le tableau Grafana « SOC / SIEM ». Il croise trois sources : le journal du WAF (blocages, limitations, adresses sources), les événements de sécurité des services (échecs de connexion, rejets de validation, signatures de webhook invalides) et les alertes du détecteur. Chaque échec de connexion indique si le compte visé existe, sans jamais journaliser les identifiants saisis. Cela suffit à distinguer un bourrage d'identifiants, qui vise surtout des comptes inconnus, d'une attaque ciblée sur un compte réel.

8.3 Consigne additionnelle 1 : authentification faciale et voyant connecté

L'accès protégé demande trois étapes dans l'ordre : identifiant, reconnaissance faciale, mot de passe. La reconnaissance (`services/face-auth`) utilise deux modèles OpenCV légers qui tournent sur CPU : YuNet détecte le visage et SFace en tire un gabarit de 128 valeurs. Le visage est comparé uniquement aux gabarits enregistrés pour l'identifiant saisi, soit une vérification 1:1, et la correspondance est acceptée à partir d'une similarité cosinus de 0,363. Ce parcours protège le back-office et, depuis le 22 octobre, la connexion du site de dons : le visage est enregistré à l'inscription et la page de connexion affiche l'état du voyant.

En cas d'échec, l'identifiant est effacé et l'utilisateur dispose d'un seul nouvel essai. Au second échec, l'accès est bloqué 5 minutes, puis 10, puis 20 à chaque nouveau blocage. Le serveur refuse l'étape du mot de passe (403) si la reconnaissance n'a pas réussi dans les 120 secondes qui précèdent, ce qui empêche de sauter l'étape depuis le navigateur. Le mot de passe est vérifié par auth-service, qui reste seul à manipuler les secrets.

Un ESP8266 connecté au hotspot affiche l'état sur une LED bicolore : vert clignotant quand le visage est reconnu, vert fixe 30 secondes quand l'accès est accordé, rouge fixe 30 secondes si le visage n'est pas reconnu, dix clignotements rouges si le mot de passe est faux. Le service n'envoie qu'un état et le microcontrôleur gère lui-même la temporisation, de sorte qu'une requête perdue ne fige pas le voyant.

Les gabarits faciaux sont des données biométriques, donc sensibles au sens de l'article 9 du RGPD, même ici où elles sont fictives. Nous ne stockons aucune photo, seulement le gabarit ; les images de la caméra sont traitées en mémoire puis oubliées ; le gabarit est supprimé avec l'habilitation ; le service reste interne à l'hôte. L'extension au site de dons fait entrer des personnes extérieures dans ce traitement : en conditions réelles, il faudrait un consentement explicite et une connexion sans biométrie au choix. Trois écarts sont assumés et décrits dans `docs/governance/biometrie-rgpd.md` : pas de détection du vivant (une photo présentée à la caméra peut tromper le modèle), caméra capturée par le navigateur plutôt que par un pilote sur l'hôte, environnement de développement libre au lieu de CLion.

9. Analyse de risques EBIOS RM

Nous avons suivi les cinq ateliers de la méthode, avec le niveau de détail utile pour un démonstrateur. La gravité va de 1 (mineure) à 4 (critique) et la vraisemblance de V1 (peu vraisemblable) à V4 (quasi certaine).

9.1 Atelier 1 : cadrage et socle de sécurité

Nous retenons quatre valeurs métier : encaisser un don (VM1), garantir l'exactitude des transactions et du classement (VM2), protéger les données des donateurs (VM3) et réserver l'administration aux opérateurs habilités (VM4). Les biens supports sont les services de la figure 1, la base MariaDB, le poste hôte et son moteur Docker, le hotspot, et pour VM4 le service d'authentification faciale et sa base de gabarits.

Événement redouté	Valeur	Impacts	G
ER1. Les dons sont impossibles pendant plusieurs minutes	VM1	Collecte perdue en pleine campagne, image dégradée, incident majeur au sens de DORA.	3
ER2. Des transactions sont faussées (payées sans paiement, statut erroné)	VM2	Fraude, classement faux, perte de confiance du partenaire de paiement.	4
ER3. La base des donateurs fuit	VM3	Violation de données personnelles à notifier, atteinte à l'image.	3
ER4. Un tiers prend la main sur le back-office	VM4	Toutes les autres valeurs deviennent atteignables.	4
ER5. Les gabarits biométriques fuient	VM4	Donnée sensible qu'on ne peut pas changer comme un mot de passe.	3
ER6. La supervision est aveuglée	VM1, VM2	Les autres événements passent inaperçus.	2

Le socle de sécurité reprend les règles d'hygiène de base : cloisonnement réseau, exposition minimale, journalisation centralisée, secrets hors du code, authentification forte pour l'administration.

9.2 Atelier 2 : sources de risque

Source de risque	Objectif visé	Pertinence
Cybercriminel organisé	Gain financier : fraude au paiement, revente de données.	Élevée
Hacktiviste	Faire tomber le site pendant la campagne Octobre Rose pour lui nuire.	Élevée
Fraudeur opportuniste	Tester des cartes volées, gonfler un classement.	Moyenne
Initié ou opérateur malveillant	Détourner le back-office, extraire des données.	Moyenne

L'attaquant du laboratoire joue tour à tour les trois premiers profils. L'initié n'est pas simulé, mais c'est lui qui justifie l'authentification forte du back-office.

9.3 Atelier 3 : scénarios stratégiques

L'écosystème compte peu de parties prenantes : le prestataire de paiement (simulé ici), les opérateurs du back-office, le poste hôte qui porte toute la plateforme et les fournisseurs de composants (images Docker, paquets npm et Python). Quatre scénarios stratégiques en découlent.

- SS1. Un hacktiviste sature le paiement pendant la campagne (ER1).
- SS2. Un cybercriminel fausse des transactions en forgeant ou en rejouant des confirmations de paiement (ER2).
- SS3. Un cybercriminel compromet des comptes de donateurs par bourrage d'identifiants, puis aspire historiques et classement (ER3).
- SS4. Un initié, ou un attaquant qui a obtenu le mot de passe d'un opérateur, tente d'entrer avec une photo de sa victime (ER4, ER5).

Le risque lié aux fournisseurs de composants est identifié mais peu traité dans ce prototype, en dehors du choix d'images officielles dont la version est indiquée.

9.4 Atelier 4 : scénarios opérationnels

Scénario	Mode opératoire	Point de détection	V
SO1 (SS1)	Flood concurrent sur le paiement depuis trois adresses (`availability`).	429 au WAF ; débit, latence p95 et part de la première adresse dans le détecteur.	V3
SO2 (SS3)	Bourrage d'identifiants sur `/api/auth/login` (`auth-abuse`).	`login_failed` au SOC ; taux d'échecs d'authentification.	V3
SO3 (SS3)	Aspiration du classement à haute fréquence (`scraping`).	Concentration du trafic sur une route et une adresse.	V3
SO4 (SS3)	Lecture massive de l'historique avec des comptes valides (`exfiltration`).	Octets sortants par fenêtre.	V2
SO5 (SS2)	Injection dans les champs de connexion et de paiement (`injection`).	403 du WAF, rafale de 400.	V2

Scénario	Mode opératoire	Point de détection	V
SO6 (SS2)	Webhook forgé ou rejoué vers `/api/donations/webhook`.	`webhook_invalid_signature` au SOC.	V1
SO7 (SS4)	Photo de l'opérateur devant la caméra et mot de passe volé.	Non détecté par le modèle ; blocage après deux échecs.	V2

Les cinq premiers scénarios correspondent aux scripts de `redteam/attacks.py` et ont été joués. SO6 et SO7 sont analysés mais pas scriptés.

9.5 Atelier 5 : traitement du risque

Risque	Avant	Mesures	Après	Risque résiduel
ER1 par SO1	3 / V3	Limitation par adresse au WAF, alerte en 10 à 15 s, redémarrage automatique.	3 / V2	Une attaque répartie sur beaucoup d'adresses contourne la limite par adresse.
ER2 par SO5, SO6	4 / V2	Requêtes paramétrées, validation, HMAC comparé en temps constant, WAF.	4 / V1	Le corps signé n'a ni horodatage ni nonce : un webhook capturé peut être rejoué, sans effet tant que le statut ne change pas.
ER3 par SO2 à SO4	3 / V3	bcrypt, message d'erreur neutre, limitation au WAF, détection sur les échecs et le volume sortant.	3 / V2	Pas de verrouillage de compte dans auth-service ; une extraction lente reste sous le seuil.
ER4 par SO7	4 / V2	Trois facteurs, blocage progressif, contrôle côté serveur.	4 / V2	Pas de détection du vivant.
ER5	3 / V2	Aucune photo stockée, service interne, suppression à la révocation.	3 / V1	Fichier de gabarits non chiffré sur l'hôte.
ER6	2 / V2	WAF comme source principale, redémarrage automatique de la chaîne.	2 / V2	Le WAF et l'hôte restent des points uniques.

Les risques résiduels les plus lourds sont ER2 et ER4, de gravité 4. Nous les acceptons pour le démonstrateur. Une mise en production demanderait au minimum un horodatage signé dans le webhook et une détection du vivant pour l'authentification faciale.

10. PCA / PRA

10.1 Activités critiques

L'activité critique est l'encaissement des dons. Elle repose sur la chaîne WAF, payment-api, psp-simulator et MariaDB. auth-service pèse moins qu'il n'y paraît : un don anonyme avec une simple adresse e-mail reste possible quand l'authentification tombe, ce qui offre un mode dégradé acceptable. La chaîne de supervision (Promtail, Loki, Prometheus, Grafana, détecteur) n'est pas nécessaire pour encaisser, mais sans elle les incidents deviennent invisibles. L'assistant RAG n'est pas critique.

10.2 Objectifs de reprise

Service	Incident	RTO visé	RPO	Mécanisme
Encaissement	Saturation par quelques adresses	Service maintenu pour les autres donateurs ; normal moins d'une minute après la fin de l'attaque	0	Limitation au WAF ; retour à RESOLVED après 3 fenêtres normales.
payment-api, auth-service	Arrêt ou plantage	1 min	0	Redémarrage automatique ; transactions validées en base.
psp-simulator	Redémarrage	1 min	Sessions ouvertes perdues ; dons concernés laissés en `pending`	Redémarrage automatique ; le donateur relance son don.
MariaDB	Arrêt du conteneur	2 min	0	Redémarrage automatique, contrôle de santé, journal InnoDB.

Service	Incident	RTO visé	RPO	Mécanisme
MariaDB	Perte du volume	Non couvert automatiquement	Depuis l'initialisation	PRA manuel : recréer le schéma depuis `infra/db/init/`.
Supervision	Arrêt d'un composant	2 min	Journaux de la coupure perdus	Redémarrage automatique ; le détecteur recrée son modèle s'il manque.
Assistant RAG	Ollama indisponible	Sans objectif	Sans objet	Démarre quand même et répond quand Ollama revient.

Ces RTO sont des objectifs. Seul le délai de détection (10 à 15 s) a été mesuré ; les durées de redémarrage dépendent du PC Hôte et se vérifient pendant la répétition de la démonstration.

10.3 Scénarios couverts et non couverts

La reprise automatique couvre l'arrêt d'un conteneur, la saturation par un petit nombre d'adresses, les attaques applicatives arrêtées par le WAF et la validation, et le redémarrage complet de l'hôte, après lequel Docker relance les conteneurs. Elle ne couvre pas la perte du PC Hôte (il n'existe pas de second site), la perte du volume de la base, une attaque répartie sur un grand nombre d'adresses, ni une compromission avérée, qui exige une investigation avant toute remise en service.

10.4 Responsabilités

Acteur	Rôle dans la reprise
WAF et Docker	Absorber la charge, redémarrer les services.
Détecteur	Détecter, qualifier la catégorie, confirmer le retour à la normale (RESOLVED).
Opérateur de l'équipe	Suivre Grafana et le SOC, valider la reprise en refaisant un don, appliquer le runbook, décider de l'escalade vers un coach.
Assistant RAG	Expliquer l'alerte et retrouver la procédure, sans aucune action sur le système.
Équipe	Analyse après incident, ajustement des règles du WAF, réentraînement du modèle.

10.5 Risques résiduels après reprise

Trois points restent ouverts. Le PSP ne relance pas un webhook qui échoue : si payment-api est arrêté au moment de la confirmation, la transaction reste `pending` alors que le PSP la considère payée, et seul un rapprochement permet de corriger. Un redémarrage du PSP fait perdre les sessions en cours. Enfin, toute la plateforme repose sur un seul hôte.

11. NIS2 et DORA

Xijin Pink jouerait le rôle d'un établissement de paiement. Pour une entité financière, le texte de référence en matière de risque informatique et de notification des incidents est DORA, le règlement (UE) 2022/2554, applicable depuis le 17 janvier 2025. La directive NIS2 (UE) 2022/2555 le reconnaît comme acte sectoriel : pour ces entités, ses obligations équivalentes s'effacent devant DORA. Elle reste néanmoins le cadre général de la cybersécurité européenne et continue de s'appliquer à d'autres acteurs de la chaîne, comme les fournisseurs de services numériques. Notre prototype a été construit en quatre jours ; nous séparons donc ce qu'il démontre de ce qu'il ne peut pas démontrer.

Exigence	Ce que le démonstrateur montre	Ce qu'il ne peut pas montrer
DORA, gestion du risque TIC (art. 5 à 16)	Cartographie des actifs et des flux (figure 1, EBIOS RM), protection (WAF, validation, secrets hors du code), détection des activités anormales demandée par l'article 10, procédures de réponse (runbooks).	Politique validée par l'organe de direction, sauvegardes testées (art. 12), revue annuelle du cadre.
DORA, incidents (art. 17 à 23)	Alertes horodatées, classées par catégorie et sévérité, journaux corrélés : la matière d'une qualification et d'une chronologie.	Classification selon les critères réglementaires et notification à l'ACPR : 4 h après le classement en incident majeur et au plus tard 24 h après la détection, rapport intermédiaire sous 72 h, rapport final sous un mois.
DORA, tests de résilience (art. 24 à 27)	Campagne d'attaques scriptée, rejouable après chaque modification.	Tests de pénétration fondés sur la menace (TLPT) menés par des testeurs indépendants.

Exigence	Ce que le démonstrateur montre	Ce qu'il ne peut pas montrer
DORA, prestataires tiers (art. 28 à 44)	Dépendance au PSP identifiée et documentée (sessions, webhook).	Contrats, registre d'information, stratégie de sortie.
NIS2, mesures de l'article 21	Analyse de risques, gestion des incidents, continuité, authentification multifacteur, cryptographie (bcrypt, HMAC).	Sécurité de la chaîne d'approvisionnement, formation, sécurité liée aux ressources humaines.
NIS2, gouvernance (art. 20) et notification (art. 23)	Les horodatages permettent de tenir l'alerte précoce sous 24 h, la notification sous 72 h et le rapport final sous un mois.	Engagement et formation de la direction, relation avec le CSIRT.

Posture de conseil

Si nous devions conseiller la direction de Xijin Pink, nous lui proposerions les arbitrages suivants, par ordre de priorité, chacun appuyé sur un élément vérifiable de la démonstration.

1. Garder la détection par apprentissage, qui a repéré les cinq catégories d'attaque en moins de 15 secondes, mais exiger une mesure des fausses alertes par heure avant de lui confier des actions automatiques.
2. Mettre en place une sauvegarde chiffrée de la base, restaurée à blanc régulièrement. C'est la seule façon de donner un RPO à la perte du volume (DORA, art. 12).
3. Ajouter la relance du webhook et un rapprochement quotidien entre le PSP et payment-api, pour qu'aucune transaction ne reste en suspens après un incident.
4. Passer au jeu de règles OWASP CRS et prévoir une protection contre les attaques réparties en amont du WAF.
5. Chiffrer les secrets (SOPS ou Vault) et fermer tous les accès d'administration anonymes.
6. Écrire la procédure de notification : qui qualifie l'incident, qui notifie, dans quels délais. Les horodatages des alertes fournissent déjà la chronologie.

12. Contributions de l'équipe

Membre	Rôle	Réalisations visibles dans le dépôt
Julien Demogue	Services et API	Structure du dépôt et architecture de départ ; auth-service, payment-api et psp-simulator (Node.js, OpenAPI, tests) ; schéma MariaDB ; intégration de l'authentification faciale et du voyant dans les pages d'inscription et de connexion du site.
Louis Marec	IA et données	Détecteur d'anomalies (indicateurs, modèle à deux étages, évaluation, scorer en ligne) et assistant RAG ; télémétrie des services et chaîne Loki, Prometheus, Grafana ; campagnes red team et export des jeux de données ; WAF et tableau SOC ; service d'authentification faciale et firmware ESP8266 ; scripts de démarrage.
Eric Girard	Infrastructure	Orchestration Docker Compose (réseau interne, volumes, contrôle de santé MariaDB, phpMyAdmin) ; conteneurisation du frontend ; configuration CORS des services ; documentation d'infrastructure.
Noé Filippi et Sterenn	Frontend	Application React et Tailwind : accueil, don, retour de paiement, classement, historique, profil, connexion et inscription ; formulaire de paiement par carte de test ; branchement aux API.

Les réalisations sont tirées de l'historique Git du projet (pull requests 1 à 8).

13. Limites et suites

Quelques points restent ouverts, et nous préférons les nommer.

- Le détecteur transmet ses alertes à `REMEDIATION_URL`, mais l'automate qui doit les recevoir n'est pas encore dans le dépôt ; les réponses par catégorie sont pour l'instant décrites dans les runbooks.
- Le rôle anonyme de Grafana a les droits d'administration. Il doit passer en simple lecture avant d'exposer le tableau sur le hotspot.
- Le frontend est servi par le serveur de développement Vite ; une image de production servirait des fichiers statiques.
- Pas de sauvegarde planifiée de la base ni de relance du webhook (section 10).
- Le taux de fausses alertes du détecteur sur du trafic normal n'est pas encore chiffré.
- Pas de détection du vivant pour l'authentification faciale.

Annexe A. Renvois vers le dépôt

Chemin	Contenu
`README.md`, `scripts/README.md`	Démarrage, vérification et arrêt de la plateforme.
`docs/architecture.md`	Composants, flux, matrice d'exposition.
`docs/payment-flow.md`	Parcours de paiement et webhook signé.
`docs/api/telemetry-contract.md`	Ce que chaque service doit journaliser et exposer.
`docs/api/alert-contract.md`	Format des alertes et réponse attendue par catégorie.
`docs/runbooks/`	Procédures par type d'incident (corpus de l'assistant RAG).
`docs/governance/`	EBIOS RM, PCA/PRA, NIS2/DORA, biométrie et RGPD, source de ce dossier.
`infra/docker/docker-compose.yml`	Orchestration de toute la plateforme.
`infra/db/init/01-schema.sql`	Schéma de la base.
`infra/observability/`	Configuration de Prometheus, Loki et Promtail, tableaux Grafana.
`services/waf/`	Règles du WAF et chemin de migration vers l'OWASP CRS.
`services/auth-service/`, `payment-api/`, `psp-simulator/`	Services métier et leurs tests.
`ai/anomaly-detection/` (voir `DOMAIN.md`)	Conception, code, tests et scorer en ligne du détecteur.
`ai/rag-assistant/`	Assistant opérateur.
`redteam/`, `tools/`	Scénarios d'attaque, trafic normal, export des jeux de données.
`services/face-auth/`, `iot/esp8266-led/`	Authentification faciale, back-office et voyant connecté.
`frontend/`	Site de dons.

Annexe B. Démarrer et vérifier la plateforme

Prérequis : Docker avec le démon lancé. Ollama est facultatif et ne sert qu'aux réponses de l'assistant RAG.

```
./scripts/start.sh           # Linux, macOS, Git Bash  (Windows : .\scripts\start.ps1)
./scripts/verify.sh          # contrôle des 11 points de santé
./scripts/stop.sh            # arrêt (--volumes pour effacer aussi les données)
```

Une fois la plateforme démarrée :

Adresse	Usage
`http://192.168.137.1:8080`	Site de dons, via le WAF
`http://192.168.137.1:3000/d/xijin-resilience`	Tableau de supervision (avant, pendant, après)
`http://192.168.137.1:3000/d/xijin-soc`	Tableau SOC / SIEM
`http://localhost:3005`	Back-office avec authentification faciale (sur l'hôte)
`http://localhost:3020`	Assistant RAG (sur l'hôte)

Pour rejouer l'incident depuis le PC Attaquant :

```
XIJIN_PAY=http://192.168.137.1:8080 python redteam/campaign.py --only availability
```

Pour reproduire l'apprentissage du détecteur sur données réelles, la procédure complète (campagne, export Loki, entraînement, évaluation) est décrite dans `ai/anomaly-detection/README.md`.